Assignment No. 03

Machine Learning

Fall 2026

Deadline: 10 November 2026

Classifying Heartbeats from ECG Signals
(a neural network built and trained from scratch in PyTorch)

Targeted Course Learning Outcome

CLO4: Design solutions using machine learning models with supervised or unsupervised techniques to address engineering challenges. **(C6)**.

Submission Instructions:

- This is a group assignment.
- Each group should be exactly 2 members.
- **Plagiarism is strictly prohibited. Plagiarism among groups would be checked with tools.**
- **Please do not use any AI tool for direct copy / paste and do not copy from others.**
- **AI plagiarism using tools such as ChatGPT, Claude, Gemini, Copilot, Cursor, etc. will be marked straight zero.**
- **Submission carries no marks. Marks will be awarded based on viva.**
- Write a Medium Blog and a LinkedIn Post and submit their links.
- Develop the front end, and push all code to GitHub with commits from both team members. Submit Repo Link.
- Use Colab or Kaggle for GPU access.

Read the whole manual before you begin.

Contents

1 Overview **3**
- 1.1 The problem . . . 3
- 1.2 Dataset . . . 3
- 1.3 Learning outcomes . . . 3

2 Instructions **4**
- 2.1 Task 1 – Load and prepare the data . . . 4
- 2.2 Task 2 – Build the network . . . 4
- 2.3 Task 3 – Loss and a gradient check . . . 5
- 2.4 Task 4 – Train . . . 5
- 2.5 Task 5 – Test once . . . 6
- 2.6 Task 6 – Analysis . . . 6
- 2.7 Task 7 – Front end and communication . . . 6
- 2.8 Required repository layout . . . 7

3 Evaluation **7**
- 3.1 Metrics . . . 7
- 3.2 Correctness checks . . . 7

4 Results **8**
- 4.1 Table 1 – Data . . . 8
- 4.2 Table 2 – Network and training . . . 8
- 4.3 Table 3 – Learning-rate choice . . . 8
- 4.4 Table 4 – Test results . . . 8
- 4.5 Figures . . . 8

Appendix: Starter Code **10**

1. Overview

1.1 The problem

An electrocardiogram (ECG) records the heart's electrical activity through electrodes on the skin. Each heartbeat produces a characteristic waveform, and abnormal beats (arrhythmias) have different shapes. Cardiologists and bedside monitors must recognise them quickly. In this assignment you will build a neural network that looks at one heartbeat and decides its type:

Input	one heartbeat: 187 samples of the ECG signal (125 samples per second)
Output	one of 5 classes: N, S, V, F or Q (table below)

and answer one practical question:

The question

How well does a small neural network recognise *each* type of heartbeat, and is overall accuracy the right number to trust when one class is far more common than the others?

You will build a fully connected neural network **from scratch in PyTorch**: you create the weight matrices yourself, write the forward pass, the softmax cross-entropy loss and the mini-batch gradient descent update with momentum and weight decay. PyTorch's automatic differentiation computes the gradients for you, and you will check one of them by hand.

1.2 Dataset

ECG Heartbeat Categorization Dataset (Kaggle): single heartbeats cut from the MIT-BIH Arrhythmia Database (PhysioNet), resampled to 125 Hz and padded with zeros to a fixed length.

Download	https://www.kaggle.com/datasets/shayanfazeli/heartbeat
Files used	`mitbih_train.csv` and `mitbih_test.csv`
Not used	`ptbdb_normal.csv`, `ptbdb_abnormal.csv`

Each file has **no header row**. Every row is one heartbeat with 188 numbers: columns 0–186 are the signal, column 187 is the class label.

Label	Class	Train rows	Test rows
0	N – normal beat	72,471	18,118
1	S – supraventricular ectopic beat	2,223	556
2	V – ventricular ectopic beat	5,788	1,448
3	F – fusion beat	641	162
4	Q – unknown or paced beat	6,431	1,608
	Total	**87,554**	**21,892**

The dataset already comes with a separate test file; you will carve a validation set out of the training file. A loader and a plotting helper are given in the Appendix; everything after loading is your work.

1.3 Learning outcomes

1. Turn a raw CSV file of signals into tensors and split it correctly.
2. Build a neural network from weight matrices: layers, ReLU, softmax.

3. Initialise weights with the He rule and explain why.
4. Implement softmax cross-entropy and check a backpropagation gradient by hand.
5. Train with mini-batch gradient descent, momentum and L2 weight decay.
6. Detect overfitting with a validation set, and evaluate a classifier on imbalanced classes.

2. Instructions

Complete the tasks in order. Every task lists exactly what must appear in your repository.

Allowed libraries

Allowed: `torch` tensors and automatic differentiation (`loss.backward()`), `nn.Module`, `nn.Parameter`, `nn.ParameterList`, `torch.logsumexp`, `torch.softmax` (for reporting probabilities only), `numpy`, `pandas`, `matplotlib`, and a front-end framework of your choice.

Not allowed anywhere: `nn.Linear`, `nn.Sequential`, `nn.ReLU` or any other ready-made layer, anything from `torch.optim`, `nn.CrossEntropyLoss`, `F.cross_entropy`, `nn.init`, `scikit-learn`, Keras or TensorFlow.

2.1 Task 1 – Load and prepare the data

1.1 Download the two MIT-BIH files from Kaggle into `data/` and load them with the loader in the Appendix. Print the shape of each.

1.2 Separate the signal (columns 0–186) from the label (column 187). Check what type the label column has and convert it to whole numbers 0–4.

1.3 Report the minimum and maximum signal value in the training file. The signals are already scaled; use them as they are.

1.4 Report the number of rows per class in both files and check them against the table in Section 1.

1.5 Run `plot_beats.py` and add the figure to your report. Under it, describe in two or three sentences how the five beat types look different.

1.6 **Split** the training file into train and validation with exactly this code, so every group has the same split:

```
rng = np.random.default_rng(42)
idx = rng.permutation(len(train_df))
n_val = int(0.1 * len(train_df))
val_idx, train_idx = idx[:n_val], idx[n_val:]
```

Report the number of rows in train, validation and test.

1.7 Convert the signals to `torch.float32` tensors and the labels to `torch.long` tensors.

2.2 Task 2 – Build the network

Use the notation of the lecture notes: layer ℓ has weight matrix $W^{[\ell]}$ (shape $n^{[\ell]} \times n^{[\ell-1]}$) and bias $b^{[\ell]}$, and

$$z^{[\ell]} = W^{[\ell]} a^{[\ell-1]} + b^{[\ell]}, \qquad a^{[\ell]} = g\big(z^{[\ell]}\big), \qquad a^{[0]} = x.$$

Use exactly this architecture:

Layer	Inputs	Units	Activation
1 (hidden)	187	128	ReLU
2 (hidden)	128	64	ReLU
3 (output)	64	5	none (logits); softmax inside the loss

2.1 Write a class `ECGNet(nn.Module)` that creates every $W^{[\ell]}$ and $b^{[\ell]}$ as an `nn.Parameter` yourself.

2.2 **He initialisation:** $W_{ij}^{[\ell]} \sim \mathcal{N}\Big(0,\ \sqrt{2/n^{[\ell-1]}}\Big)$ (second argument = standard deviation), and $b^{[\ell]} = 0$. Call `torch.manual_seed(0)` before creating the network.

2.3 Write `forward(x)` for a mini-batch x of shape $(B, 187)$. With one example per row, a layer becomes $Z^{[\ell]} = A^{[\ell-1]}W^{[\ell]\top} + b^{[\ell]}$. Write ReLU yourself, $g(z) = \max(z, 0)$, using `torch.clamp(z, min=0)`. Return the logits $z^{[3]}$, shape $(B, 5)$.

2.4 Print the total number of trainable parameters and show the calculation by hand in your report.

2.3 Task 3 – Loss and a gradient check

3.1 **Softmax cross-entropy.** For logits $z \in \mathbb{R}^5$ and true class y,

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{5} e^{z_j}}, \qquad L = -\log \text{softmax}(z)_y = -z_y + \log \sum_{j=1}^{5} e^{z_j}.$$

Write `cross_entropy(logits, y)` returning the mean of L over the batch. Use the right-hand form with `torch.logsumexp`; computing e^z directly can overflow.

3.2 **Gradient check.** Take the first 64 training rows. Run a forward pass, compute the loss and call `loss.backward()`. Then compute the output-layer gradient yourself:

$$\frac{\partial J}{\partial W^{[3]}} = \frac{1}{B}\left(\hat{Y} - Y\right)^{\top} A^{[2]}, \qquad \frac{\partial J}{\partial b^{[3]}} = \frac{1}{B}\sum_{i=1}^{B}\left(\hat{y}^{(i)} - y^{(i)}\right),$$

where $\hat{Y}$ $(B \times 5)$ holds the softmax probabilities, Y $(B \times 5)$ holds the one-hot labels, and $A^{[2]}$ $(B \times 64)$ holds the second hidden layer's activations. Report the largest absolute difference from the gradients PyTorch stored in the `.grad` of your $W^{[3]}$ and $b^{[3]}$.

2.4 Task 4 – Train

Write the update yourself (lecture notes, Sections 20.7 and 20.8). For every parameter P keep a velocity v_P, starting at zero. After `loss.backward()`, inside `torch.no_grad()`:

$$g = \frac{\partial J}{\partial P} + \lambda P \ \text{ (weights only; for biases } g = \tfrac{\partial J}{\partial P}\text{)}, \qquad v_P = \beta\, v_P + (1-\beta)\, g, \qquad P = P - \alpha\, v_P.$$

Use exactly these settings:

Batch size B	128	Momentum β	0.9
Weight decay λ	10^{-4}	Epochs	30
Learning rate α	chosen from $\{0.01,\ 0.1,\ 0.5\}$	Shuffle	every epoch

4.1 Write `train(alpha)` that trains a fresh network (seed 0) for 30 epochs on the training rows. After every epoch record the training loss, the validation loss, the training accuracy and the validation accuracy.

4.2 **Choose α.** Run `train` for each of the three values. Plot validation loss against epoch for all three on one figure. Choose the α with the lowest validation loss.

4.3 For the chosen α, plot training and validation loss against epoch on one figure, and training and validation accuracy on another. Save the parameters from the epoch with the **lowest validation loss** with `torch.save(model.state_dict(), ...)`.

2.5 Task 5 – Test once

5.1 Load the saved parameters and predict the class of every test row: the class with the largest logit.

5.2 Write `accuracy`, `confusion_matrix` and `per_class_accuracy` yourself (definitions in Section 3).

5.3 Report test accuracy, the 5×5 confusion matrix, the per-class accuracy of each of the five classes, and the balanced accuracy.

2.6 Task 6 – Analysis

Answer in the blog and in `results/analysis.md`:

6.1 A model that always answers "N" would get what test accuracy? Compute it from the class table. Compare it with your model's accuracy. What does this tell you about using accuracy alone?

6.2 Which class has the lowest per-class accuracy? Using the confusion matrix, which class is it most often mistaken for, and why might those two beat types be hard to separate?

6.3 Look at your loss curves. Is there a gap between training and validation loss? Is the network overfitting, underfitting, or neither? Explain from the curves.

6.4 Why did you not initialise every weight to zero? Why does the He rule use $\sqrt{2/n^{[\ell-1]}}$ for ReLU layers? (Lecture notes, Section 20.2.)

6.5 What does momentum do to the update when two consecutive mini-batch gradients point in opposite directions? What does weight decay do to a weight that receives no gradient?

6.6 The F class has only 641 training rows. Suggest one change to the *data* or the *loss* that could help the network on rare classes. Explain the idea; do not run it.

2.7 Task 7 – Front end and communication

7.1 A web interface (Streamlit, Gradio, Flask, ...) where the user picks a test-set row number (or uploads a CSV file with one row of 187 values), sees the heartbeat plotted, and sees *your* network's predicted class and the probability of all five classes as a bar chart. Screenshot in the README.

7.2 **Medium blog** (800–1500 words): the problem, the example beats, how the network was built and trained, the results table, and your answers to Task 6. At least two figures.

7.3 **LinkedIn post** linking the blog and repository.

7.4 **GitHub repository**: public; README with run instructions; meaningful commits from *both* members.

2.8 Required repository layout

`data/`	the two CSV files (do not commit them; they are too large)
`src/load_data.py`	the given loader
`src/plot_beats.py`	the given plotting helper
`src/prepare.py`	Task 1
`src/model.py`	`ECGNet`, `cross_entropy`
`src/grad_check.py`	Task 3.2
`src/train.py`	Task 4 (update rule, α choice, curves, saved model)
`src/metrics.py`	the three metric functions of Task 5.2
`src/test_model.py`	Task 5
`results/`	figures, tables, `analysis.md`, saved model
`app/`	Task 7 front end
`README.md`	how to run everything

3. Evaluation

3.1 Metrics

Training: the mean softmax cross-entropy J on the training and validation rows.

Test: with m test rows, true labels $y^{(i)}$ and predictions $\hat{y}^{(i)}$:

Accuracy	$\frac{1}{m}\sum_i \mathbf{1}[\hat{y}^{(i)} = y^{(i)}]$
Confusion matrix	C_{jk} = number of rows whose true class is j and predicted class is k; rows are true classes, columns are predictions
Per-class accuracy	for class j: $C_{jj}/\sum_k C_{jk}$, the share of class-j beats that were recognised as class j
Balanced accuracy	the average of the five per-class accuracies; every class counts equally, however rare

Expected range

A correctly implemented network of this size usually reaches about 96–98% test accuracy. Per-class accuracy is much lower for the rare classes (S and F) than for N, so balanced accuracy will be clearly below overall accuracy. Overall accuracy below 90% means a bug (check that the label column is not among the inputs, and that the loss decreases). If the network predicts only class N, check your learning rate and your update rule.

3.2 Correctness checks

Before submitting, confirm:

- The input has 187 columns; the label column is not among them.
- The network has 32,645 trainable parameters.
- With freshly initialised weights the loss is close to $\log 5 \approx 1.61$ (every class roughly equally likely).
- Your hand-computed $\partial J/\partial W^{[3]}$ and $\partial J/\partial b^{[3]}$ match PyTorch's to within 10^{-5}.

- Weight decay is applied to the weights and not to the biases.
- Your confusion matrix entries add up to 21,892, and each row adds up to that class's test count.
- The test rows are never used for training, for choosing α or for choosing the epoch.

4. Results

Present the following in `results/` and in the blog, using these layouts.

4.1 Table 1 – Data

Train / validation / test rows
Minimum / maximum signal value
Rows per class, train file (N, S, V, F, Q)
Rows per class, test file (N, S, V, F, Q)

4.2 Table 2 – Network and training

Trainable parameters
Initial loss (before training)
Gradient check: max difference for $W^{[3]}$ / $b^{[3]}$
Chosen α
Best epoch (lowest validation loss)
Training / validation loss at best epoch
Training / validation accuracy at best epoch
Training time and hardware

4.3 Table 3 – Learning-rate choice

α	**Lowest validation loss**	**Validation accuracy at that epoch**
0.01		
0.1		
0.5		

4.4 Table 4 – Test results

	N	**S**	**V**	**F**	**Q**
Per-class accuracy					

Test accuracy
Balanced accuracy
Always-"N" accuracy (Task 6.1)

Also include the full 5×5 test confusion matrix with class names on both axes.

4.5 Figures

1. One example beat per class (Task 1.5).

2. Validation loss against epoch for the three learning rates (Task 4.2).
3. Training and validation loss, and training and validation accuracy, for the chosen α (Task 4.3).
4. Test confusion matrix as a heat-map.
5. Front-end screenshot.

Appendix: Starter Code

This is the only code provided: a loader that reads the two CSV files as they are, and a helper that plots one beat per class. Neither converts, splits or cleans anything.

A.1 Environment

```
pip install torch numpy pandas matplotlib
```

A.2 Getting the data

1. Create a free Kaggle account and open the dataset page (link in Section 1).
2. Click **Download**, unzip, and place `mitbih_train.csv` and `mitbih_test.csv` in `data/`.

A.3 src/load_data.py

```
from pathlib import Path
import pandas as pd

FILES = {"train": "mitbih_train.csv", "test": "mitbih_test.csv"}

def load_raw(data_dir="data"):
    """Return {'train': DataFrame, 'test': DataFrame} exactly as
       stored.

    The files have no header row: columns 0..186 are the heartbeat
    signal and column 187 is the class label. Nothing is converted,
    split or cleaned here.
    """
    data_dir = Path(data_dir)
    frames = {}
    for split, name in FILES.items():
        path = data_dir / name
        if not path.exists():
            raise FileNotFoundError(f"Missing {path}. Download the
               dataset first.")
        frames[split] = pd.read_csv(path, header=None)
    return frames

if __name__ == "__main__":
    raw = load_raw()
    for split, df in raw.items():
        print(f"{split}: {df.shape[0]} rows, {df.shape[1]} columns")
    print(raw["train"].iloc[:3, [0, 1, 2, 185, 186, 187]].to_string()
       )
```

A.4 src/plot_beats.py

```
import matplotlib.pyplot as plt
from load_data import load_raw

CLASS_NAMES = ["N: normal", "S: supraventricular ectopic", "V:
   ventricular ectopic",
               "F: fusion", "Q: unknown / paced"]
FS = 125                                    # sampling frequency in Hz
```

```python

def plot_one_per_class(df, out_path="results/example_beats.png"):
    """Plot the first beat of each class in df (raw DataFrame from
       load_raw)."""
    fig, axes = plt.subplots(1, 5, figsize=(16, 3), sharey=True)
    time_ms = [1000 * i / FS for i in range(187)]
    for k, ax in enumerate(axes):
        row = df[df[187] == k].iloc[0]
        ax.plot(time_ms, row.iloc[:187].values)
        ax.set_title(CLASS_NAMES[k], fontsize=9)
        ax.set_xlabel("time (ms)")
    axes[0].set_ylabel("normalised amplitude")
    fig.tight_layout()
    fig.savefig(out_path, dpi=150)
    print(f"saved {out_path}")

if __name__ == "__main__":
    import os
    os.makedirs("results", exist_ok=True)
    plot_one_per_class(load_raw()["train"])
```

Run `python src/load_data.py` and then `python src/plot_beats.py` from the repository root to confirm the files load.

Good luck. A monitor that misses the rare beat is the one that matters.